

SANKUR KUNDU

Software Engineer | Backend & Full-Stack

Bhubaneswar, India | +91 97761 94201 | sankur.kundu.tw@gmail.com
linkedin.com/in/sankur-kundu | github.com/Shakhtar-Sankur | shakhtar-sankur.github.io

SUMMARY

Software engineer (B.Tech CSE, 2026) who builds systems and proves them correct: **quorumdb**, a distributed SQL database in Rust verified by deterministic simulation and **TLA+**, and **faultline**, a Jepsen-style tester run against real **etcd** clusters. As Gigzen's co-founder, shipped alone a **three-app delivery platform** on one PostgreSQL backend with **row-level security**.

EXPERIENCE

Gigzen — Co-founder & Founding Engineer

Apr 2026 – Present

Bhubaneswar, India · Early-stage startup (incorporation in progress) · Sole engineer across every product

- **Sole engineer** on **Waggle**, a local-delivery platform of **three Android apps** (Waggle Gig for riders, Waggle Business for shops, Waggle for customers) built from one **TypeScript / React / Capacitor** codebase on a shared **Supabase PostgreSQL** backend: **53 tables, 90 RLS policies, 154 SQL functions**.
- Put the business rules in the database, where no client can bypass them: **first-accept-wins dispatch** (45-second offer window), **4-digit handover codes**, **Razorpay/UPI** payments reconciled by UTR, **GST invoices** and **pg_cron** billing runs.
- Built real-time logistics: nearby-job matching, live rider tracking with **snap-to-road** map matching, voice turn-by-turn with rerouting, a distance-based live ETA, and parcel delivery (Waggle Send).
- **Security-tested** the platform as one system: **150 automated attacks** on the database access rules, all blocked; a **42-step Playwright** end-to-end run across all three apps, from rider verification to delivery rating.
- Earlier shipped **Waggle 1.4** (open source): **43 languages** across **92 countries**, offline-first messaging, phone numbers hidden at the database level.
- Built **Populace**, an open-source multi-user testing tool (**Node.js, zero runtime dependencies**) shipped as an **npm** CLI, a **GitHub Action** and an **Electron** desktop app. Its **first run found 5 defects in 3.5 minutes** after full manual testing had passed, including one that silently broke **every new signup**.
- Designed its **13-method adapter contract** and proved it on third-party software (**Gitea: 1,106 API calls**); **3.17M API calls** across 30–300 simulated users with **zero API failures**; **122 self-tests** in CI.

Freelance Data Scientist — Upwork, remote (client in Spain)

Jun 2026 – Jul 2026

- Merged two Spanish hotel censuses (**5,909** and **8,058 records**) from Excel and STATA sources with Pandas.
- Built a Google Trends panel of **10,038 city-months**: deseasonalised, z-scored and flagged for positive demand shocks.

PROJECTS

quorumdb — Rust, zero dependencies: a distributed SQL database, PostgreSQL wire-compatible

- Built from scratch a **distributed SQL database** that **psql** connects to: an **LSM storage engine**, **Raft consensus**, a **sharded multi-Raft store** that splits and rebalances itself, **serializable ACID transactions** (Percolator) and a SQL planner; **1,500 tpmC** on a **TPC-C**-derived benchmark, consistency checks passing.
- Proved by **deterministic simulation** (**10,000 Raft clusters**, **610K crashes**, histories machine-checked) and a **TLA+** model (**337K states**); found **6 real bugs**, catches all **15 planted**; **time-travel debugger** rewinds runs by replay.
- Profiling TPC-C led to a **block cache**, **prefix bloom filters**, lazy MVCC version reads and **LIMIT pushdown**; CI boots a real 3-node server, runs a **psql** session, restarts it and checks the data survived.

faultline — Rust, zero dependencies: Jepsen-style fault injection for etcd

- Runs real **etcd** clusters in **Linux network namespaces**, injects **iptables partitions**, **kill -9** crashes and **SIGSTOP** pauses, and checks histories for **linearizability**, **watch completeness** and **lost updates**.
- Tested **4 etcd release lines** with **1.26M operations** under **288 faults**: linearizability and watches held; traced **lost updates** under etcd locks to a paused leader applying a timed-out write **6 s** late, and showed **fenced writes** prevent it.
- Every checker must catch a **planted bug** (stale serializable reads, duplicated or skipped watch events, unfenced locks); every run saves its full history and an **HTML report** of latency and throughput through each fault; CI tests real etcd.

SKILLS

CS Fundamentals: Data Structures & Algorithms, System Design, Distributed Systems, DBMS internals, OS, Networks

Languages: Rust, TypeScript, JavaScript, Python, SQL (PL/pgSQL), Java

Backend & Data: PostgreSQL, Supabase, Row-Level Security, pg_cron, REST APIs, Node.js, FastAPI, Flask, Redis, Razorpay/UPI

Frontend & Mobile: React, Capacitor (Android), Electron, Leaflet/MapLibre maps, HTML, CSS

Testing, DevOps & Cloud: Simulation testing, fault injection, TLA+, Playwright, pytest, GitHub Actions, Docker, Kubernetes, GCP, AWS, Linux

EDUCATION

B.Tech, Computer Science and Engineering (AI & ML)

Institute of Engineering and Management (IEM), Kolkata

Sep 2022 – Apr 2026